CIVIQ

Hyperlocal Civic Accountability Platform

Product Requirements Document

Version	1.0 — Hackathon Build
Target	India — Civic Issue Reporting Platform
Model	Community-first, Independent of Government APIs
Stack	React Native · Supabase · Gemini AI · Cloudflare R2

1. Overview

Civiq is a mobile-first platform that enables citizens to report, validate, track, and collectively drive the resolution of hyperlocal civic issues — potholes, water leakages, broken streetlights, waste management failures, and public infrastructure damage.

The platform operates independently of government APIs, using community upvoting, public accountability dashboards, and auto-escalation to drive accountability without requiring official integration. Government adoption becomes a Phase 2 outcome driven by data and public pressure — not a prerequisite.

Problem Statement

Problem	Current State	Impact
Fragmented reporting	WhatsApp groups, verbal complaints	No tracking, no accountability
No verification layer	Single unverified reports ignored	Low resolution rates
Zero transparency	Issues disappear after reporting	Citizen distrust
No data aggregation	No ward-level insight	No predictive maintenance
Digital divide	Platforms require high literacy	Excludes vulnerable communities

Goals

- Citizens can report an issue in under 60 seconds
- AI auto-categorises and deduplicates reports without user effort
- Community validation creates verified, high-credibility issue records

- Public accountability layer creates pressure without government API dependency
- Ward-level dashboards produce open data for journalists, NGOs, and future government integration

2. Users & Roles

Role	Description	Key Actions
Citizen (Reporter)	Any resident with a smartphone	Report, upvote, verify resolution
Citizen (Verifier)	Neighbours who confirm issues	Upvote, add photo proof, mark resolved
Ward Champion	Power user, top contributor	Moderate, verify issue resolutions, export data
Admin	Platform operator	Manage categories, review flags, view analytics

3. Feature Set

3.1 Onboarding

Phone-based authentication	Core

OTP login via mobile number. No email required. Hindi and English language selection at first launch.

- Phone number → OTP → verified account in 3 steps
- Ward auto-detected via GPS on first login, manually adjustable
- Language preference stored in user profile

3.2 Issue Reporting

Quick report flow	Core

Floating action button triggers report flow. Citizen captures photo or video, AI runs in background, location is pinned, description is optional (voice note supported).

- Photo / video capture or gallery upload
- AI analyzes the submitted image to generate issue category, severity, tags, civic summary, and recommended department.GPS pin with map confirmation — citizen can drag pin to correct location
- Voice-to-text description (Hindi / English)
- Report queued if offline, submitted on reconnect

3.3 AI Categorisation

Gemini 2.0 Flash (Current Implementation)	Core

A single Gemini 2.0 Flash API call analyzes each submitted image to classify the issue, estimate severity, generate a structured civic summary, and recommend the appropriate authority for escalation.

- Detects issue category: **Pothole, Water Leakage, Streetlight, Waste, Road Damage, Flooding, Other**
- Estimates severity on a **1 (Minor) to 5 (Critical Safety Hazard)** scale

- Generates descriptive tags and confidence score
- Produces a concise AI-generated civic summary for authority escalation
- Suggests the responsible department (e.g., Public Works Department, Municipal Health Department, Water Authority)
- Returns structured JSON:

```
{
  "category": "...",
  "severity": 4,
  "confidence": 0.94,
  "tags": [],
  "summary": "...",
  "recommended_department": "...",
  "risk_level": "Medium",
  "estimated_priority": "High"
}
```

- Reports are published immediately while AI analysis completes asynchronously (typically within 30 seconds)
- Supports up to **1,500 requests/day** using the Google AI Studio free tier

3.4 Duplicate Detection

Three-signal duplicate engine	Core

Every new report triggers a parallel check across three signals, producing a 0–100 confidence score that determines merge, flag, or publish.

- Signal 1 — Geo-radius (50m): PostGIS ST_DWithin query on open issues of same category (weight: 40%)
- Signal 2 — Visual similarity: pgvector cosine similarity on Gemini image embeddings (weight: 40%)
- Signal 3 — Category + text match: keyword overlap scoring (weight: 20%)
- Score 0–40: unique issue, published immediately
- Score 41–80: flagged for community review with side-by-side comparison
- Score 81–100: High confidence duplicate → Reporter reviews possible duplicate → Merge (OR) Create New Report

3.5 Community Validation

Neighbour upvote + verification	Core

Issues gain credibility through community confirmation. Structured upvoting, resolution verification via photo proof, and community-controlled status transitions.

- Upvote button: confirms issue exists, increments severity score
- Severity score formula: base_severity × log(upvotes + 1) × age_weight
- Status flow: Open → Escalated → In Progress → Pending Verification → Verified Resolved → Closed
- Resolution verification: citizen submits 'after' photo, 3+ community votes to confirm
- Dispute: reporter or verifier can reopen a closed issue with evidence

3.6 Accountability Engine

SLA timer + auto-escalation	Core

The primary differentiator. Issues progress through a transparent accountability pipeline without any government API. Escalation generates a structured civic report and automatically emails the responsible ward authorities and municipal offices.

Optional channels: WhatsApp (if verified public contact exists), PDF export, Public issue page.

- Day 0 — SLA timer starts at submission

- Day 7 — no action: 'Unacknowledged' badge appears on issue
- Day 14 — AI-generated civic report emailed to relevant authorities
- Day 30 — 'Overdue' ward badge added to public dashboard
- Day 60 — issue included in Monthly Civic Performance Report (PDF) (journalist-ready)
- Ward Performance Dashboard:
  - Open issues
  - Resolved issues
  - Average resolution time
  - Most common issue categories
  - Overdue issue count

3.7 Map & Discovery

Live issue map	High

Interactive map view of all open issues in the user's ward and surrounding area. Heatmap overlay shows issue density by zone.

- Mapbox or Google Maps SDK with clustered markers
- Filter by: category, severity, status, date range
- Tap marker: issue card with photo, upvote count, SLA timer, status
- Heatmap toggle: density overlay for ward-level hotspot view
- List view alternative for low-end devices

3.8 Impact Dashboard

Ward-level analytics	High

Public dashboard accessible without login. Displays resolution rates, category breakdowns, average response times, and predictive hotspot indicators.

- Ward scorecard: open / resolved / overdue counts
- Category trend chart: issue type over last 12 months
- Avg days to resolution per ward and category
- Predictive hotspot: wards with recurring same-category issues flagged pre-season
- Open data export: CSV download, public API endpoint (no auth required)

3.9 Future Enhancements

Points, badges, leaderboard	High

Lightweight engagement layer. Points for meaningful actions only — not for passive browsing. Ward leaderboard resets weekly to keep competition fresh.

- Points: Report (+10), Upvote confirmed issue (+3), Verify resolution (+15), Issue resolved that you reported (+20)
- Badges: Road Warrior (10 potholes), Water Watch (5 water issues), Streak (7-day active)
- Ward leaderboard: top 10 reporters this week, all-time ward hero
- Push notification: 'Your report was resolved' + points awarded
- WhatsApp status update opt-in for resolution notifications

4. Technical Architecture

4.1 Frontend

Layer	Technology	Rationale
Mobile app	React Native (Expo)	Single codebase for Android + iOS; Expo Go for rapid hackathon demo
Maps	react-native-maps (Google Maps)	Native performance; free tier sufficient for MVP
State management	Zustand	Lightweight, no boilerplate vs Redux
UI components	NativeWind (Tailwind for RN)	Fast styling, consistent design tokens
Camera	expo-camera + expo-image-picker	Gallery + capture in one package
Offline queue	expo-sqlite (local queue)	Reports stored locally if offline, synced on reconnect
Push notifications	Expo Notifications	Free, cross-platform, no separate service needed

4.2 Backend

Layer	Technology	Rationale
API / BaaS	Supabase	PostgreSQL + Auth + Realtime + Storage in one; free tier covers MVP
Database	PostgreSQL 15 (via Supabase)	ACID, relational, mature, free
Geospatial	PostGIS extension	Native geo queries (ST_DWithin for 50m radius scan)
Vector search	pgvector extension	Image embedding storage + cosine similarity in same DB; no separate vector DB
Realtime	Supabase Realtime (websockets)	Live map updates, status change notifications
Background jobs	Supabase Edge Functions (Deno)	AI tagging queue, SLA timer checks, escalation triggers
File storage	Cloudflare R2	Zero egress fees; S3-compatible, Free tier for MVP (10 GB storage); pay-as-you-go beyond free tier.
Escalation	Resend email, Optional WhatsApp Business API	Auto-escalation to councillors via public contact

4.3 AI & External APIs

API	Purpose	Cost
Google Gemini 2.0 Flash (AI Studio)	Image categorisation, severity scoring, tag extraction, embedding generation	Free — 1,500 req/day
Google Maps Platform	Map rendering, geocoding, reverse geocoding	Free — $200 credit/month

Twilio / WhatsApp Business *(Optional)*	Future notification channel	Free tier — 1,000 msg/month
Resend	Automated authority notifications	Free — 3,000 emails/month

4.4 Authority Directory Management

Aspect
Authority contact information is maintained independently of issue reports. • Admins can import ward authority data through CSV. • Admins can manually edit authority details. • Contact records include: - Ward Number - Municipality - Councillor Name - Councillor Email - Ward Office Email - Assistant Engineer Email - Last Updated - Source (Manual Entry / CSV Import / Government Dataset) This directory is used by the escalation engine to identify the correct authority for each reported issue.

4.5 Database Schema

Core tables

Table	Key Columns	Notes
users	id, phone, ward_id, language, points, created_at	Auth managed by Supabase Auth
wards	id, ward_number, ward_name, district, municipality, authority_type, councillor_name, councillor_email, ward_office_email, assistant_engineer_email, health_inspector_email, last_updated	Seeded from India local body data
issues	id, user_id, ward_id, category, severity, status, location (GEOGRAPHY), image_url, embedding (vector), created_at, sla_deadline	PostGIS GEOGRAPHY for geo queries; pgvector for embeddings
issue_votes	id, issue_id, user_id, type (upvote/verify_resolved), photo_url, created_at	One vote per user per issue per type
issue_status_log	id, issue_id, old_status, new_status, changed_by, reason, created_at	Full audit trail
escalations	id, issue_id, channel (whatsapp/email), sent_at, response_received	Track all escalation events
badges	id, user_id, badge_type, awarded_at	Gamification records

5. Authentication

Aspect	Decision	Detail
Provider	Supabase Auth	Built-in, free, handles JWT issuance and refresh
Method	Phone OTP	SMS via Supabase's Twilio integration; no password, no email
Session	JWT (7-day expiry)	Stored securely in device keychain via expo-secure-store
Row-level security	Supabase RLS policies	Users can only edit their own reports; votes are append-only
Anonymous reporting	Not supported in v1	Requires phone to prevent spam; re-evaluate post-launch
Admin access	Role claim in JWT	admin role set manually in Supabase for platform operators

6. Key API Contracts

6.1 Issue submission flow

Step	Action	Actor
1	POST /issues — create issue record with status=pending, store image to R2	Client → Supabase
2	Supabase trigger fires Edge Function: gemini_tag	Server
3	gemini_tag calls Gemini Vision API with image URL	Edge Function → Gemini
4	Gemini returns { category, severity, tags, confidence, summary,recommended_department, risk_level, estimated_priority, embedding }	Gemini → Edge Function
5	Edge Function runs duplicate check (geo + vector + text)	Edge Function → PostGIS + pgvector
6a	Score < 40: update issue status=open, publish to map	Edge Function → DB
6b	Score 41–80: status=pending_review, notify reporter of possible duplicate	Edge Function → DB + Push
6c	Score > 80: Mark as High Confidence Duplicate, Notify Reporter, Reporter confirms: Merge OR Create New Issue	Edge Function → DB + Push

6.2 Escalation flow

Trigger	Action	Channel
Issue age = 7 days, status = Open	Add "Unacknowledged" badge and notify the reporter	Database Update + Push Notification
Issue age = 14 days, status = Open	Generate an AI Civic Report and automatically email the responsible ward authorities using the Authority Directory	Resend Email API

Issue age = 14 days, status = Open	Log escalation event and update issue status to Escalated	Database Update
Issue age = 30 days, status = Open	Display issue on the Public Accountability Dashboard and mark the ward as having overdue civic issues	Dashboard Update
Issue age = 60 days, status = Open	Include issue in Monthly Civic Performance Report	Scheduled Edge Function

7. Infrastructure & Cost

Service	Provider	Tier	Monthly Cost
Database + Auth + Realtime	Supabase	Free (500MB DB, 50k MAU)	₹0
Image / video storage	Cloudflare R2	Pay-as-you-go	~₹125
AI categorisation	Google AI Studio (Gemini)	Free (1,500 req/day)	₹0
Maps	Google Maps Platform	Free ($200 credit)	₹0
Backend hosting	Supabase Edge Functions	Free (500k invocations/month)	₹0
Email escalation	Resend	Free (3,000/month)	₹0
Total MVP	—	—	~₹125/month

8. Hackathon Build Milestones

Phase	Deliverable	Priority
Phase 1 — Core loop	Auth (phone OTP) + Issue submission (photo + GPS) + Gemini AI tagging + Map view	Must have for demo
Phase 2 — Validation	Upvoting + duplicate detection + status flow + community verification	Strong demo impact
Phase 3 — Accountability	SLA timer + AI Civic Report Generation + Authority Directory + Email Escalation + ward dashboard	Differentiator
Phase 4 — Platform Polish	Push Notifications + Offline Sync + UI/UX Improvements + Performance Optimisation	Polish
Phase 5 — Open Data & Future Enhancements	CSV Export + Public API + Journalist PDF Reports + Gamification (Badges & Leaderboards)	Bonus

9. Non-Functional Requirements

Requirement	Target	Approach

Report submission time	< 60 seconds end-to-end	Optimistic UI; AI runs async after publish
Offline capability	Report queued if no connection	expo-sqlite local queue, sync on reconnect
Map load time	< 2s for ward-level view	Clustered markers, pagination of issue data
Language support	Hindi + English	i18n via i18next, all UI strings externalised
Image compression	< 500KB per upload	Client-side compression before R2 upload
Accessibility	WCAG 2.1 AA for core flows	Large tap targets, screen reader labels, high contrast
Data privacy	No PII in public issue data	User identity hidden from public; only ward shown